Hook & Horizon



Mees Rehorst

22-6-2025

Table of contents

Hook & Horizon ----------1
Table of contents ----------1
1. The Vision for Hook & Horizon ----------2
2. My Workflow ----------2
Scrum----------2
Clean code ----------2
My adaptation to Scrum ----------2
3. Epics ----------2
4. User Stories----------2
Epic 1 - Core Fishing Mechanics ----------2
Epic 2 - Fish System & Variants ----------2
Epic 3 - Mutations & Special Effects ----------2
Epic 4 - Biomes & World Structure ----------2
Epic 5 - Boats & Land vehicles----------2
Epic 6 - Fishing Rods, Crafting & Equipment ----------2
Epic 7 - Inventory, Storage & Fast Travel ----------2
Epic 8 - Weather, Time & Global Events ----------2
Epic 9 - Enemies & Dangers----------2
Epic 10 - Exploration & Navigation----------2
Epic 11 - NPCs, Quests & Rewards----------2
Epic 12 - Stats, Logs & Leaderboards ----------2
Epic 13 - Buffs, Bonuses & Passive Effects ----------2
Epic 14 - Character Customization----------2
Epic 15 - Saving, Loading & Progress System ----------2
Epic 16 - UI & User Experience ----------2
Epic 17 - Voxel Modeling & Art Style----------2
Epic 18 - Future Expansions & DLC----------2

The Vision for Hook & Horizon

I've seen many games fall short — not because of bad ideas, but because of a lack of clear vision from the developer. Without direction, even great concepts lose their impact. That's why I've taken the time to write down exactly what I want Hook & Horizon to be.

At the same time, I believe that good games aren't made in a vacuum. That's why I truly value the input of players, fellow developers, and critics. Whether it's a gameplay tweak, a balancing suggestion, or just a wild idea — I'm always open to feedback and discussion. This vision isn't a locked plan, but a compass: it gives direction, while still allowing room to grow, evolve, and improve.

Hook & Horizon is a voxel-style open-world fishing adventure that blends exploration, creativity, and a touch of the absurd. It's not just about casting a line — it's about discovering a world built on detail, personality, and hidden depth.

Players explore a grid-based world made up of distinct, handcrafted biomes: from tropical beaches and deep oceans to icy tundra's, toxic wastelands, volcanic lakes of lava, and even a koi-filled, cherry blossom-covered Chinese biome. Each biome holds unique fish species, visual styles, ambient effects, and special biome buffs — like enhanced lure speed, increased rare fish chance, or elemental fish encounters.

Fishing itself is a core loop full of variation. Fish come in different species, sizes (from Tiny to Monster), and mutations — such as gold, rainbow, phantom, crystal, or even nuclear. Some mutations are rare and cosmetic; others affect gameplay. Your gear matters too: players can buy or craft fishing rods with unique properties like the Lava Rod (only usable in lava biomes), Soulcatcher, Electric Rod, or the Champion Rod earned through quests.

Transportation plays a big role. Boats aren't just cosmetic — each one has specific stats (speed, buff compatibility, access level) and might be required for certain biomes. From a simple canoe to a submarine, a steampunk cruiser, or even a DLC pirate ship like the Flying Dutchman, every boat feels unique. You'll also be able to travel by land using creatures like donkeys or llamas — which double as mobile inventory storage.

Speaking of inventory, players have limited space: both in number of items and in total fish weight carried, requiring thoughtful management. Fast travel is possible (however limited) via player-placed portals, adding strategy to your world traversal.

The world is alive, with changing seasons, day/night cycles, and weather events like storms, rain, and mist. Special global events — such as Golden Hour, Full Moon, Mutant Invasion, or International Fish Day — bring temporary modifiers and surprises.

It’s not all peaceful: enemies like sharks, orcas, alligators, and mythical creatures like the Kraken or the Loch Ness Monster can attack your boat and even cause you to lose progress.

Quests from NPCs help guide progression, offering rewards like unique rods, money, or transportation. Players can customize their character’s appearance — from skin tone to outfits and hats — and track their personal journey in a robust statistics screen, logging everything from most valuable fish to quests completed, biomes visited, fish mutated, and distance traveled.

There’s also room for competition, with leaderboards, rare fish to chase, and ranking systems for players who want to prove they’re the ultimate angler.

And finally, players can mark coordinates, leave personal notes, and eventually discover mysterious structures like lighthouses or hidden pirate treasure — expanding the world even further.

My vision for Hook & Horizon is to create a fishing game where the act of fishing is only the beginning. It’s about discovery, personalization, and a cozy-yet-wild world full of secrets, systems, and strange beauty.

A game I want to play — and now, finally, one I’m building.

Workflow

Clean code

One of the biggest challenges I expect to face during this project is writing clean, maintainable code. As a junior developer, it's tempting to just "make it work" — but I know that messy code becomes a major problem as a project grows.

That's why I'm actively trying to apply clean code principles throughout the development of Hook & Horizon. I aim to:

- Use clear and consistent naming conventions for variables, classes, and methods
- Keep functions short, focused, and readable
- Structure my code using the SOLID principles, even if only partially at first
- Avoid duplicating logic, and instead build modular, reusable components
- Write scripts that are easy to understand later — even by future me

Clean code isn't just about perfectionism — it's about saving time, reducing bugs, and making it easier to expand or debug the game in the long term. Even though I'm still learning, I want to build this game with habits that will help me become a better developer overall.

UI/UX Design

Information Architecture & Flow

- Menu Architecture: Mapping the entire hierarchy of the main menu, pause screens, and sub-menus to ensure the fewest clicks possible to reach settings or "play game".
- Onboarding Flows: Designing the logical steps of the initial setup, terms of service agreements, and character creation screens.
- Save/Load Systems: Creating safe, error-free dialog loops that confirm when a player is about to overwrite or delete game data.

Player Guidance & In-Game Systems

- HUD Layout Optimization: Positioning critical real-time data—like health bars, mini-maps, and ammo counts—to prevent visual clutter while keeping vital information within the player's peripheral vision.
- Quest Tracking & Wayfinding: Designing objective trackers, on-screen compass markers, or in-world glowing paths that guide players toward their next goal without breaking immersion.
- Inventory Management Systems: Defining how players sort, filter, view stats, and equip items within complex gear menus (e.g., grid systems vs. list layouts).

Visual & Interactive Feedback.

- State Management: Creating visual variations for buttons and icons to reflect different states, such as active, hovered, pressed, locked, or disabled.
- Diegetic UI Integration: Collaborating with 3D artists (claude aswell) to integrate UI elements directly into the game world, like an interactive health display on a or a physical map held in a player's hands.
- Signaling and Feedback: Designing screen flashes, damage indicators, and cooldown overlays that tell the player exactly what is happening to their avatar in real-time.

Controls & Cross-Platform Adaptation

- Button Mapping Layouts: Configuring default control schemes and customization screens for keyboards, mice, gamepads, and custom accessibility controllers.
- Responsive Layout Design: Ensuring that text scales correctly and menus shift fluidly across different screen aspect ratios, such as ultrawide monitors, TVs, and mobile screens.
- Touch & Gesture Control Mapping: Defining tap zones, swipe behaviors, and virtual joysticks specifically for mobile devices.

Accessibility & Localization

- Color Blindness Adaptation: Implementing alternative color palettes and distinct iconography so that color-blind players can easily tell teams or item rarities apart.
- Text Scaling and Fonts: Selecting legible typefaces and providing slider controls for text sizing, closed captioning background opacity, and speaker labels.
- Localization Guardrails: Designing menus with dynamic text containers that gracefully expand to accommodate longer languages (like German or Russian) without clipping or breaking the layout.

1. Epics

What is an Epic?

An Epic is a large, high-level feature or goal in a project. It's too big to complete all at once, so it’s broken down into smaller User Stories that can be worked on individually.

1. **Core Fishing Mechanics:**
   - The full gameplay loop: casting, waiting, reaction timing, catching, and feedback.
2. **Fish System & Variants:**

- Managing fish types, weight classes, rarity levels, and the internal fish database.

3. **Mutations & Special Effects:**
   - Unique traits like burning, phantom, rainbow, nuclear, and their gameplay impact.
4. **Biomes & World Structure:**
   - The game world built from voxel-based biomes (beach, deep ocean, swamp, volcanic, etc.) — each with unique fish, buffs, and visual style.
5. **Boats & Land Vehicles:**
   - Various boats and mounts with different speeds, effects, biome access, and inventory capabilities.
6. **Fishing Rods, Crafting & Equipment:**
   - Buyable and craftable rods, rod bonuses, and special rods unlocked via quests (e.g., Lava Rod, Champion Rod).
7. **Inventory, Storage & Fast Travel:**
   - Weight and item limits, inventory management, storage locations, and portal-based fast travel.
8. **Weather, Time & Global Events:**
   - Day/night cycles, seasonal changes, weather (rain, mist), and special timed events like Golden Hour, Full Moon, or Mutant Invasion.
9. **Enemies & Dangers:**
   - Hazards like sharks, orcas, alligators, and sea monsters (e.g. the Kraken) that can destroy boats or affect progress.
10. **Exploration & Navigation:**
    - World exploration, biome traversal, coordinate markers, map integration, and personal notes.
11. **NPCs, Quests & Rewards:**
    - Interacting with villagers, quest chains, and rewards like unique gear, gold, or vehicles.
12. **Stats, Logs & Leaderboards:**
    - Detailed stat tracking (fish caught, largest catch, mutation count, gold earned, etc.) and optional competitive rankings.
13. **Buffs, Bonuses & Passive Effects:**
    - Status effects like Lure Speed, Luck, or biome-specific modifiers tied to gear or time-based events.
14. **Character Customization:**
    - Changing player appearance: gender, skin tone, outfits, hats — possibly with unlockable cosmetics.
15. **Saving, Loading & Progress System:**
    - Game save/load system, including inventory, quests, player location, and long-term progression.

16. **UI & User Experience:**
    - Inventory interface, stats screen, fish catch popups, shop UI, crafting menus, and clean navigation.
17. **Voxel Modeling & Art Style:**
    - Creating modular voxel-style visuals for environments, boats, fish, props, and UI elements.
18. **Future Expansions & DLC:**
    - Ideas Planned content for later updates: The Flying Dutchman, Lighthouse system, Pirate Treasure, Steampunk biome, and more. (Will make sure the game won't feel ‘pay to win’ by making these skins you can apply to your boats!)

2. User Stories

For each Epic, I break the feature down into User Stories — short, goal-oriented descriptions from the player's point of view. They help me stay focused on what the player needs rather than just technical tasks.

Each User Story follows the format:

"As a [developer/user], I want [something], so that [benefit]."

This format keeps my development grounded in purpose and player experience. Along with each story, I define:

- Definition of Ready – What conditions must be true before I start
- Definition of Done – When is this feature truly finished
- Tasks – The steps needed to implement the story
- Story Points – An estimate of how much effort it will take (1–13 Fibonacci scale)

By using this structure, I can better plan, prioritize, and track progress — even as a solo developer.

Epic 1 - Core Fishing Mechanics

"As a player, I want to cast my fishing rod into the water so I can attempt to catch a fish."

Definition of Ready:

- Water zones exist and are detectable
- Input system is functional
- Player has a rod equipped

Definition of Done:

- Player can cast when near water
- A line or float appears visibly
- Basic animation or visual feedback is present
- Rod cannot cast on land

Task	Story Points
Set up input for casting	2
Detect proximity to water	1
Instantiate float/line on cast	2
Basic placeholder animation or line arc	2
Prevent casting on land	1

"As a player, I want to wait for a fish to bite and react within a short time window."

Definition of Ready:

- Casting mechanic is working
- Timer logic is available

Definition of Done:

- After casting, a random timer starts
- A visual/sound cue indicates a bite
- Player has limited time (e.g. 2 seconds) to press a key to catch
- Failure to react results in no fish

Task	Story Points
Implement random timer	2
Add bite alert (sound/visual)	2
Create input window for reaction	3
Define success/fail logic	2

"As a player, I want to see what kind of fish I caught, including its name, weight, and mutation (if any)."

Definition of Ready:

- Fish data model is available
- Reaction logic is working

Definition of Done:

- Catch triggers a randomly selected fish
- Weight is generated based on size class
- Mutation is applied based on chance
- Catch info is displayed in a popup or UI panel

Task	Story Points
Create fish database (species list)	3
Implement weight & size class generator	2
Add mutation logic (basic chance system)	3
Design simple catch popup UI	2
Display catch info on success	2

"As a player, I want to hear sound effects and see simple visual feedback during each step of fishing."

Definition of Ready:

- Base loop is working

Definition of Done:

- SFX for casting, bite, catch, and fail
- Visual feedback (splash, flash, etc.)
- Enhances overall feedback for the player

Task	Story Points
Implement casting sound	1
Add splash visual effect	2
Bite alert sound & icon	1
Success/fail sound	1

Summary for Epic 1:

User Story	Story Points
Casting mechanic	8 SP
Reaction system	9 SP
Catch result system	12 SP
Feedback (SFX/VFX)	5 SP

Epic 2 - Fish System & Variants

"As a developer, I want to create a flexible fish database so I can manage all fish species and their properties."

Definition of Ready:

- Fish attributes (name, weight range, rarity, biome, mutation chance) are planned
- Unity project is set up with basic architecture

Definition of Done:

- A database (e.g., ScriptableObject, JSON, or custom data class) exists
- Fish entries include species name, base weight, rarity level, allowed biomes
- Data can be accessed in-game by other systems

Task	SP
Design FishData structure or ScriptableObject	3
Add fields for species name, weight, biome list, rarity	2
Create 10–15 initial fish entries	2
Make database accessible to fishing system	2

"As a player, I want each fish I catch to have a randomized weight and size classification."

Definition of Ready:

- Weight ranges per fish are defined
- Catch event is implemented

Definition of Done:

- Each fish instance gets a randomized weight within bounds
- Based on % deviation, fish gets a size label (Tiny, Normal, Huge, Monster, etc.)
- Label is shown in UI

Task	SP
Add random weight generation logic	2
Define weight thresholds per size category	2
Implement size classification system	2
Show size in catch popup	1

"As a player, I want different fish to appear in different biomes so that exploring the world matters."

Definition of Ready:

- Biomes exist and are identifiable by the game
- Fish database includes biome tags

Definition of Done:

- Each biome has its own list of possible fish
- Catch logic references current biome to select fish
- Different biomes give visibly different fish results

Task	SP
Assign biome tags to fish data	1
Implement biome-based fish filtering in catch logic	3
Create test pools for at least 3 biomes	2

"As a player, I want some fish to be rarer than others, so that finding a rare one feels exciting."

Definition of Ready:

- Fish rarity levels are defined (e.g., Common, Uncommon, Rare, Legendary)
- Catch system is functional

Definition of Done:

- Each fish has a rarity value (probability-based)
- Rarity affects spawn chance
- Rarity is displayed in catch popup (e.g., color-coded or tagged)

Task	SP
Add rarity value to fish data	1
Implement rarity-based selection logic	2
Show rarity indicator in UI	1
Balance rarity curve for MVP fish pool	2

Summary for Epic 2:

User Story	Story Points
Flexible fish database	9 SP
Random weight + size classification	7 SP
Biome-specific fish selection	6 SP
Rarity system	6 SP

Epic 3 - Mutations & Special Effects

"As a player, I want some fish to have mutations so that every catch has the chance to be surprising or valuable."

Definition of Ready:

- Fish base system is working
- Mutation types are defined
- Random system is in place

Definition of Done:

- Fish can spawn with 0 or more mutations
- Mutation is chosen from a defined pool
- Mutation name appears in the catch UI

Task	SP
Define mutation types (names, rarity, effects)	2
Add mutation field to fish data	1
Create mutation roll logic (based on % chance)	2
Show mutation in fish popup	1

"As a player, I want different mutations to affect fish stats or behavior, so they aren't just visual."

Definition of Ready:

- Mutation data includes effect type
- Fish generation supports dynamic attributes

Definition of Done:

- Mutations can apply effects such as:
    - Bonus gold
    - Extra weight
    - Increased catch difficulty
    - Environmental restrictions (e.g., nuclear only at night)
- Effects are reflected in catch output

Task	SP
Expand mutation data structure with effect types	2
Link mutation to stat changes during fish generation	3
Implement at least 3 functional effects	3
Reflect mutation effect in catch description	1

"As a developer, I want to create a scalable mutation system so that I can easily add more mutations in the future."

Definition of Ready:

- Basic mutation logic is working
- Fish creation pipeline supports injection/modular logic

Definition of Done:

- Mutations use a modular system (e.g., a dictionary of mutation effect functions or ScriptableObject system)
- Adding new mutations requires minimal code edits
- Clean and reusable structure

Task	SP
Design scalable mutation system architecture	3
Refactor basic system to use modular logic	4
Test adding 2 new mutations dynamically	2

"As a player, I want mutated fish to have unique visual indicators so I can instantly recognize them."

Definition of Ready:

- Fish visuals exist (prefab or 2D image)
- Mutations have defined visual types

Definition of Done:

- Each mutation adds a visual effect (e.g., glow, color tint, particle)
- Visual appears in-game and/or in catch UI
- Placeholder visuals acceptable for MVP

Task	SP
Assign visual types to each mutation	2
Add shader/tint/FX system to fish prefab	3
Show mutation visual on caught fish	1

Summary for Epic 3:

User Story	Story Points
Basic mutation logic	6 SP
Mutation gameplay effects	9 SP
Scalable system for future mutations	9 SP
Visual indicators for mutated fish	6 SP

Epic 4 - Biomes & World Structure

"As a player, I want to explore distinct biomes with their own look, so the world feels diverse and worth exploring."

Definition of Ready:

- Core movement and camera systems work
- Biome list is defined
- World loading system planned

Definition of Done:

- Each biome has its own visual style
- Biomes can be explored via player movement
- Placeholder terrain is acceptable in MVP
- At least 2–3 biomes implemented

Task	SP
Define biome list and visual identifiers	1
Create basic terrain/prefabs per biome	3
Implement biome-switch logic (chunk/grid system or scene swap)	4
Place biomes in world map (manual or procedural)	2

"As a player, I want each biome to have its own unique fish pool so that exploring new areas leads to new catches."

Definition of Ready:

- Fish database includes biome tags
- Catch system supports contextual logic

Definition of Done:

- Each biome defines a list of available fish
- Catch system references current biome

- Biome-based fish are noticeably different

Task	SP
Assign biome tag to each fish	1
Implement biome-checking in fishing logic	2
Create test pools for at least 3 biomes	2
Add a fallback/default biome for error handling	1

"As a player, I want each biome to offer unique buffs or conditions, so that each area feels mechanically distinct."

Definition of Ready:

- Buff types are defined
- Buff system framework is planned

Definition of Done:

- Biomes can assign passive buffs (e.g., +Rare Chance, +Lure Speed)
- Buffs apply automatically when player enters biome
- UI indicator shows active biome buff

Task	SP
Define list of biome buffs and effects	1
Implement buff trigger on biome enter	3
Show current biome buff in UI	2
Add cooldown/removal logic for leaving biome	1

"As a developer, I want the biome system to be modular so I can easily add or tweak biomes over time."

Definition of Ready:

- Existing biomes are implemented via prefabs or templates
- Chunk loading or zone tracking is working

Definition of Done:

- Biomes are prefabs, scenes, or data structures that can be reused
- Adding a new biome doesn't require hardcoding fish, visuals, or logic
- Supports expansion via data/configs

Task	SP
Set up biome prefab or data-driven system	3
Refactor hardcoded systems into biome-driven logic	4
Test adding one new biome dynamically	2

Summary for Epic 4:

User Story	Story Points
Visual & traversable biomes	10 SP
Biome-based fish logic	6 SP
Buffs per biome	7 SP
Modular biome system	9 SP

Epic 5 - Boats & Land vehicles

"As a player, I want to board and disembark a boat so I can travel across water freely."

Definition of Ready:

- Player movement and animations exist
- A basic boat prefab is available in scene
- Input system supports an "Interact" action

Definition of Done:

- When standing next to a boat and pressing Interact, the player parents to the boat and gains boat-movement controls
- Pressing Interact again disembarks the player at boat's edge and restores normal movement
- A UI prompt ("Press E to board") appears when in range

Task	SP
Add "Interact" input binding in Input Actions	1
Create trigger collider & Board/Disembark logic	3
Swap control scripts between Player and Boat	3
Display/hide on-screen prompt via Range Trigger	2

"As a player, I want each boat type to have different speed and buffs so I can choose the best vessel for each biome."

Definition of Ready:

- BoatData ScriptableObject type exists (name, speed, buff, allowed biomes)
- At least two boat prefabs available

Definition of Done:

- A `BoatData` SO defines speed multiplier & buff (e.g. +10% lure speed)
- Each boat prefab references its SO
- Boarding that boat applies its speed & buff to the fishing controller
- Current boat stats display in a HUD element when aboard

Task	SP
Create `BoatData` SO class and assets (2 boats)	3
Reference BoatData on boat prefabs	1
Modify boat-movement script to apply speed & buff	3
Add simple HUD panel showing "Speed: X Buff:Y"	2

"As a player, I want to mount a land vehicle (e.g. donkey) with extra inventory space so I can travel overland and carry more items."

Definition of Ready:

- A mount prefab exists (e.g. donkey with collider)
- Inventory system placeholder exists

Definition of Done:

- Interact with mount to board; press Interact to dismount
- While mounted, movement speed increases by mount's speed stat
- Player's max inventory capacity increases by mount's bonus
- Mount and dismount resets when vehicle is destroyed or left

Task	SP
Create `MountData` SO for speed & inventory bonus	2
Add mount prefab and trigger-based boarding logic	3
Apply mount stats to player movement & inventory cap	3
Ensure dismount returns player to correct location	1

Summary for Epic 5:

User Story	Story Points
5.1 Board & disembark boats	9 SP
5.2 Boat types with unique stats & buffs	9 SP
5.3 Mounts with speed & inventory bonus	9 SP

Epic 6 - Fishing Rods, Crafting & Equipment

"As a player, I want to buy fishing rods with gold at a shop so that I can access better gear."

Definition of Ready

- Shop UI exists (placeholder)
- Player currency ("gold") tracked
- Player needs a working inventory (I'll allow logged)

Definition of Done

- Shop lists available rods with price, name, icon
- Player can click "Buy" if they have enough gold
- Gold is deducted and rod is added to their inventory
- Confirmation/feedback is shown

Task	SP
Extend RodData SO to include `price` field	2
Build basic Shop UI panel with scroll list	3
Implement buy button logic & gold-check	3
Hook into player inventory to add purchased rod	2
Play purchase SFX and UI feedback	1

"As a player, I want to craft fishing rods from gathered materials so I can create unique gear."

Definition of Ready

- Material items exist in inventory system
- Crafting UI panel stub exists

Definition of Done

- Crafting recipe table defined (e.g. Bamboo + Rope → Bamboo Rod)
- Crafting UI lists recipes, required ingredients, and "Craft" button
- On craft: ingredients removed, new rod added to inventory, success/fail feedback

Task	SP
Create `RecipeData` SO (ingredients + result RodData)	3
Design Crafting UI with recipe list & requirement checks	3
Implement `CanCraft()` validation and `Craft()` action	3
Integrate ingredient removal & rod addition to inventory	2

Add crafting SFX and success/failure popups	1

"As a player, I want to earn special rods through quests so I feel rewarded for challenges."

Definition of Ready

- Quest system exists with reward hooks
- QuestData SO can specify a RodData reward

Definition of Done

- Quests can award a `RodData` reference
- On quest completion, the special rod is added to inventory
- Distinct UI notification for "Quest Reward: [RodName]"

Task	SP
Extend `QuestData` SO to include `RodData` reward slot	2
Update quest-completion handler to grant rod reward	2
Create at least two special `RodData` assets for quests	1
Design a simple "Quest Reward" popup in UI	2
Play a fanfare SFX on reward	1

"As a player, I want each rod to have unique bonuses (e.g. +Lure Speed, +Luck) so I can strategize my builds."

Definition of Ready

- `RodData` SO exists with fields for bonuses (e.g. `lureSpeedBonus`, `luckBonus`)
- FishingController reads rod bonuses

Definition of Done

- Each `RodData` SO exposes one or more bonus parameters
- Equipping a rod applies its bonuses to fishing logic (reaction window, rarity roll, etc.)
- UI reflects current rod and its stats in an equipment panel

Task	SP
Add bonus fields to `RodData` SO (lureSpeed, luck, etc.)	2
Equip/unequip logic in player controller	3
Modify fishing routines to factor in bonuses (e.g. shorter delays, higher rare chance)	4
Build Equipment UI panel showing current rod and its stats	3
Test and balance two example rods (e.g. Speed Rod, Luck Rod)	2

Summary for Epic 6:

User Story	Story Points
6.1 Buy Fishing Rods	11 SP
6.2 Craft Fishing Rods	12 SP
6.3 Quest-Reward Rods	8 SP
6.4 Equip Rod Bonuses	14 SP
Total	**45 SP**

Epic 7 - Inventory, Storage & Fast Travel

"As a player, I want my inventory to have weight and item count limits so that I must plan what I carry."

Definition of Ready

- Basic item/pickup system exists
- Player has an Inventory class or data structure

Definition of Done

- Inventory tracks both total item count and total weight
- Each item (fish, gear) has a weight value
- Prevent adding items when either limit is exceeded
- Player is notified (UI message) when inventory is full

Task	SP
Add `weight` field to all item SOs	2
Extend Inventory class to track weight & count	3
Implement "CanAddItem()" validation	2
Show popup/toast when inventory is full	1

"As a player, I want to deposit items in chests or docks so I can free up inventory space."

Definition of Ready

- Inventory Limits story is done
- Scene contains placeholder storage prefabs

Definition of Done

- Storage objects have a trigger-zone for deposit
- When in range, "Press E to Store" appears
- Stored items move from player inventory into a persistent storage list
- Items can be retrieved later

Task	SP
Create `Storage` component with item list	2
Add trigger collider & proximity UI prompt	2

Implement `StoreAll()` & `RetrieveAll()` APIs	2
Persist storage contents across sessions	1

“As a player, I want to place portals and warp between them so I can traverse the map quickly.”

Definition of Ready

- Scenes or zones support named waypoints
- Player can interact with objects

Definition of Done

- Player can craft/deploy/repair a portal object at current location
- “Activate” interaction lists all placed portals and current location
- Selecting a destination instantly moves the player there
- Cooldown or cost (e.g. gold) is applied

Task	SP
Create `Portal` prefab and `PortalManager` to track them	2
Implement placement logic and a limited count	2
Build interaction UI to list and select portals	1
Teleport player and apply cost/cooldown	2

“As a player, I want a visual inventory screen so I can see my items, weight, and capacity.”

Definition of Ready

- Inventory data structure is complete
- UI Canvas is available

Definition of Done

- Inventory panel lists items with icons, names, quantities, and weights
- Displays current weight/count and max values
- Supports scrolling for long lists
- Opens with a key press (e.g. “I”) and closes likewise

Task	SP
Design Inventory UI layout in Canvas	2
Populate UI list from Inventory data	3
Add weight/count bars or numeric readouts	1
Hook up open/close input binding	1

Summary for epic 7:

User Story	Story Points

Inventory Limits	8 SP
Storage Locations	6 SP
Fast Travel	5 SP
Inventory UI	7 SP
Total	26 SP

Epic 8 - Weather, Time & Global Events

"As a player, I want a continuous day/night cycle so I can experience time passing in the world."

Definition of Ready

- Scene contains a directional light representing the sun.
- Basic UI exists for overlay or clock.

Definition of Done

- World lighting slowly shifts over a 24-hour period (configurable day length).
- Shadows and ambient colors change to reflect dawn, day, dusk, and night.
- A simple UI clock or skybox indicator shows current time.
- Other systems can query the current time (e.g. fishing bite rates differ night/day).

Task	SP
Create a TimeManager to track time progression	2
Animate sun's rotation & ambient color changes	3
Design or configure four key lighting presets	1
Build a UI clock or time-of-day indicator	2

"As a player, I want changing weather (rain, storm, mist) so that fishing conditions feel dynamic."

Definition of Ready

- Particle effect prefabs exist for rain, storm, mist.
- TimeManager is implemented.

Definition of Done

- Weather spawns randomly or by schedule (configurable probabilities).
- Appropriate particle system and audio play during each weather.
- FishingController adjusts bite chance or difficulty based on weather (e.g. rain +10% bite).
- Weather state is visible in UI or sky (e.g. clouds, raindrops).

Task	SP
Build WeatherManager with random weather roll	2
Hook up rain, storm, and mist particle prefabs	2
Play matching ambient audio for each weather type	1

Integrate weather modifiers into fishing logic	2
Show weather icon/UI indicator	1

“As a player, I want seasons (spring, summer, autumn, winter) so that environments and fish pools change over time.”

Definition of Ready

- Day/Night cycle works.
- FishDatabase supports seasonal tags or a mechanism for filtering by season.

Definition of Done

- Seasons advance automatically every N in-game days.
- Each season alters environment visuals (e.g. foliage color, snow coverage).
- FishDatabase filters fish based on current season (e.g. Ice fish only in winter).
- A UI element displays current season.

Task	SP
Extend TimeManager to track days and season progression	2
Define and configure four seasonal visual presets	2
Tag FishData entries with allowed seasons	1
Update fishing logic to query current season	1
Add season icon or label to the UI	1

“As a player, I want special events (Full Moon, Fish Day, Golden Hour, Mutant Invasion) so the world feels alive and varied.”

Definition of Ready

- TimeManager, WeatherManager, Seasons are in place.
- UI notification system exists (e.g. popup or toast).

Definition of Done

- Events trigger on specific calendar dates or times (e.g. Full Moon every 7 days, International Fish Day every 30 days).
- Each event applies unique gameplay modifiers (e.g. +rare bite chance, mutant spawn buff).
- A notification appears at event start and end.
- After event ends, modifiers are removed.

Task	SP
Define event schedule and effects list	1
Implement EventManager to trigger and track events	2
Apply and remove gameplay modifiers during events	2
Create UI popup/toast notifications for start/end	1
Test at least two events (e.g. Full Moon and Fish Day)	2

Summary for Epic 8:

User Story	Story Points
8.1 Day/Night Cycle	8 SP
8.2 Weather Effects	8 SP
8.3 Seasons System	7 SP
8.4 Timed Global Events	8 SP
Total	31 SP

Epic 9 - Enemies & Dangers

"As a player, I want sharks and orcas to patrol the water and damage my boat on contact so I must stay alert."

Definition of Ready

- Boat and water zones implemented
- Enemy prefab placeholder available

Definition of Done

- Shark and orca prefabs spawn or patrol in water biomes
- On collision with the boat, they reduce boat health and play damage SFX
- A brief "Boat Damaged!" UI notification appears
- Enemies despawn or retreat after attacking

Task	SP
Create Shark & Orca prefabs with colliders/rigs	2
Implement simple patrol AI (waypoints or roaming)	3
Detect collision with boat → apply health damage	2
Play attack SFX & show UI feedback	1

"As a player, I want a swarm of piranhas in shallow water that can chew through my float or damage my boat if I linger too long."

Definition of Ready

- Shallow-water biome detection exists
- Float and boat damage systems in place

Definition of Done

- Piranha prefab spawns in groups when player casts in shallow zone
- They rapidly converge on the float or boat
- On contact, float breaks or boat takes minor damage
- A "Piranha Attack!" alert and SFX play

Task	SP
Create Piranha prefab and group spawner logic	2
Trigger swarm spawn on shallow-water cast	2
Swarm AI: move toward target & apply damage	3
Attack SFX & UI notification	1

"As a player, I want alligators and king crabs at shorelines that can damage mounts or interrupt my fishing."

Definition of Ready

- Shoreline zones are tagged in the world
- Mount system exists

Definition of Done

- Alligator and king crab prefabs spawn near shorelines
- They detect and chase either player-on-foot or mounted
- On collision, they apply damage or knock down the player/mount
- “Hazard!” SFX and UI warning appear

Task	SP
Create Alligator & King Crab prefabs with colliders	2
Shoreline spawn triggers	2
Simple chase AI & collision damage	2
Knockdown animation or effect + SFX/UI prompt	1

"As a player, I want a Kraken event that can sink my boat, forcing me to flee or use special gear."

Definition of Ready

- EventManager and boat-health system exist
- Kraken model or placeholder available

Definition of Done

- Kraken spawns during a special event (e.g. at certain locations or random)
- Tentacles strike the boat periodically, reducing health drastically
- A “Kraken Attack!” UI warning and dramatic SFX play
- If boat health reaches zero, boat sinks and the player is ejected to shore

Task	SP
Create Kraken boss prefab with tentacle colliders	3
Hook into EventManager to trigger boss spawn	2

Implement tentacle strike logic & heavy damage	4
Boat-sink animation, player ejection, SFX & UI	4

Summary for Epic 9:

User Story	Story Points
9.1 Aquatic Predators	8 SP
9.2 Piranha Swarm	8 SP
9.3 Shoreline Hazards	7 SP
9.4 Kraken Boss Event	13 SP
Total	**36 SP**

Epic 10 - Exploration & Navigation

"As a player, I want to open a full-screen world map with biome overlays so I can plan where to fish or explore."

Definition of Ready

- World coordinate system exists (even placeholder grid)
- UI Canvas is set up

Definition of Done

- Pressing the "Map" key toggles a full-screen map panel
- Map shows biome regions (color-coded) and player position as a dot
- Map can be closed with the same key

Task	SP
Design Map UI panel layout	2
Implement open/close input binding	1
Render biome overlay on map texture	1
Show player position marker on map	1

"As a player, I want to place and name markers on the map so I can remember spots for fishing or resources."

Definition of Ready

- World Map panel is functional (10.1 done)
- Basic UI input (click/tap) is implemented

Definition of Done

- Clicking (or tapping) on the map opens a "New Marker" prompt
- Player enters a name and confirms
- A marker icon and label appear at that map location

- Marker data persists between sessions

Task	SP
Detect map clicks and convert to coords	3
Create marker prefab (icon + text label)	1
Build name-entry prompt UI	2
Save/load marker data to player profile	2

"As a player, I want a small minimap and compass HUD so I always know my direction and nearby terrain."

Definition of Ready

- Map coordinate system and camera exist
- HUD Canvas is available

Definition of Done

- A corner minimap shows a top-down view of the player's immediate surroundings
- A compass displays current facing direction (N/E/S/W)
- Markers from the full map (10.2) appear on the minimap

Task	SP
Design minimap + compass HUD elements	1
Render camera to RenderTexture for minimap	3
Implement compass arrow using camera heading	2
Overlay markers onto minimap	2

"As a player, I want to jot down notes tied to coordinates so I can remember special findings later."

Definition of Ready

- Map markers (10.2) are implemented
- UI framework for panels exists

Definition of Done

- Selecting a marker or grid cell opens a text-entry field
- Player can type and save a note for that location
- Notes are listed in a "My Notes" UI panel
- Notes persist between play sessions

Task	SP
Add note-entry UI panel	1
Link notes to marker/coordinate data	1

Store notes in save system or player prefs	2
Create “My Notes” list panel	1

Summary for Epic 10:

User Story	Story Points
10.1 World Map Display	5 SP
10.2 Custom Map Markers	8 SP
10.3 Minimap & Compass	8 SP
10.4 Location Notes	5 SP
Total	**26 SP**

Epic 11 - NPCs, Quests & Rewards

“As a player, I want to talk to NPCs using a simple dialogue UI so I can learn about the world and receive quests.”

Definition of Ready

- Basic input system supports “Interact”.
- NPC prefabs exist in the scene (placeholder OK).
- UI canvas available.

Definition of Done

- Standing in range shows “Press E to talk”.
- Dialogue panel opens with NPC name, portrait (optional), and message lines.
- Next/Close buttons (or input) work; dialogue blocks player movement while open.
- Dialogue can trigger quest offers or tips.

Task	SP
Create `NPCData` (name, portrait, lines, quest hooks) as ScriptableObject	2
Add trigger collider & Interact prompt on proximity	2
Dialogue UI panel (name, body text, Next/Close)	3
Simple dialogue runner (advance lines, branching by tag)	3
SFX for open/close and typewriter effect	1

“As a player, I want a quest system (accept, track, complete) so I have clear goals and rewards.”

Definition of Ready

- Dialogue can invoke “Offer Quest”.
- Inventory, gold, and event hooks (catch fish, enter biome, etc.) exist.

Definition of Done

- `QuestData` defines id, title, description, objectives, and rewards.

- Player can Accept/Decline; active quests appear in a Quest Log.
- Objectives update via events (e.g., catch X fish).
- Completing a quest grants rewards and shows a completion popup.
- Progress persists between sessions.

Task	SP
Create `QuestData` SO (id, text, objectives, rewards)	3
Quest Log UI (list, details, tracked pin)	3
Accept/Decline + tracking state machine	5
Objective listeners (events from fishing/biome/inventory)	5
Persist active/completed quests in save system	3

"As a player, I want meaningful quest rewards (gold, items, rods) so completing quests feels valuable."

Definition of Ready

- Quest system is functional.
- Inventory and Rod systems exist.

Definition of Done

- Rewards can grant gold, items, rods, and cosmetics.
- Rewards are displayed in a clear UI message with icons.
- Inventory capacity is validated; overflow blocked with message.

Task	SP
Extend `QuestData` with reward list (gold, items, `RodData`, cosmetics)	2
Reward grant pipeline (with inventory checks & toasts)	3
Reward popup UI with icons + SFX/fanfare	2
Balance baseline reward values for early quests	3

"As a player, I want chained quests with prerequisites and light branching so storylines can progress."

Definition of Ready

- Basic quest complete flow works.
- Save/Load captures quest state.

Definition of Done

- Quests can require completion of previous quest(s).
- Branching paths supported by simple conditions (e.g., chosen response, biome visited).
- Chain progress displays in Quest Log (e.g., 2/5).
- Two example chains implemented.

Task	SP

Prerequisite system (requires completed quest ids)	3
Chain/sequence logic + progress indicator	5
Branch conditions (flag set via dialogue choice or action)	5
Author & test two simple chains	3

"As a player, I want daily/weekly bounties, so I have repeatable goals and steady income."

Definition of Ready

- Time system and events exist.
- Quest/Objective system is stable.

Definition of Done

- Bounty board shows rotating tasks (e.g., "Catch 5 Rare fish").
- Resets daily/weekly; progress tracked separately from story quests.
- Rewards scale with player level/biome.
- Simple anti-exploit checks (unique completion per period).

Task	SP
Scheduler & reset windows (daily/weekly)	3
Bounty generator templates + selection rules	5
Board UI list + claim/reward flow	2
Reward scaling curve	3

Summary for Epic 11:

User Story	Story Points
11.1 NPC Dialogue	11 SP
11.2 Quest Core (Accept/Track/Complete)	19 SP
11.3 Quest Rewards	10 SP
11.4 Quest Chains & Branching	16 SP
11.5 Daily/Weekly Bounties	13 SP
Total	**69 SP**

Epic 12 - Stats, Logs & Leaderboards

"As a player, I want my personal stats tracked (fish caught, biggest fish, gold earned) so I can see my progress."

Definition of Ready

- Fishing, inventory, and gold systems fire events.
- Save system available.

Definition of Done

- Stats model tracks totals, bests, and counts.

- Stats panel shows key metrics and highlights records.
- Stats persist across sessions.

Task	SP
Stats model & event hooks	2
Wire up increments on catch/sell/quest complete	3
Persist/load stats	2
Stats UI panel	3

"As a player, I want a Fish Log/Compendium so I can complete collections."

Definition of Ready

- Fish database exists with rarity, biome, season.

Definition of Done

- Log shows discovered species with silhouette for undiscovered.
- Entries display best size/weight, mutation badges, and caught count.
- Completion % per biome/season shown.

Task	SP
Fish Log UI (grid + details)	3
Unlock-on-first-catch & reveal entry	3
Mutation/size badges	2
Completion % and filters	1
Link entries to FishData	2

"As a player, I want a session summary screen when I quit or sleep so I can review what I achieved."

Definition of Ready

- Stats deltas computable.
- UI framework in place.

Definition of Done

- Summary lists fish caught, rarest, gold earned, quests completed.
- "New Records" called out.
- Optional screenshot/export of panel.

Task	SP
Compute delta since last session start	3
Summary UI panel	3
Export/screenshot option	2
Hook into save/quit flow	1

"As a player, I want leaderboards (local now, online-ready later) so I can compare results."

Definition of Ready

- Stats exist for metrics (e.g., biggest fish).

Definition of Done

- Local leaderboard ranks best records on device.
- Abstraction layer for future online posting.
- Basic anti-cheat flagging (range sanity checks).

Task	SP
Leaderboard model (entries, categories)	2
Local ranking + display UI	3
Async interface for future backend	5
Basic validation/sanity checks	2

Summary for Epic 12:

User Story	Story Points
12.1 Personal Stats	10 SP
12.2 Fish Log/Compendium	11 SP
12.3 Session Summary	9 SP
12.4 Leaderboards (Local + Wrapper)	12 SP
Total	**42 SP**

Epic 13 - Buffs, Bonuses & Passive Effects

"As a developer, I want a modular buff system so I can add timed and passive effects safely."

Definition of Ready

- TimeManager and biome detection exist.
- Items/rods/boats expose bonus fields.

Definition of Done

- `BuffData` SO (id, duration, stacks, modifiers).
- Apply/refresh/expire pipeline with events.
- Stacking rules: stack, refresh, or unique.
- Debug overlay lists active buffs.

Task	SP
Create `BuffData` + modifiers schema	2
Apply/expire lifecycle + timers	5
Stacking/priority policies	3
Events: OnBuffApplied/Removed for UI	2

"As a player, I want time- and biome-based buffs to apply automatically."

Definition of Ready

- Buff system in place.
- Time/biome systems emit enter/leave.

Definition of Done

- Entering biome applies its passive buff; leaving removes it.
- Golden Hour / Full Moon can apply timed buffs.
- UI shows current biome/time buffs.

Task	SP
Integrate TimeManager/biome triggers	3
Author sample buffs (RareChance, LureSpeed)	1
Apply/remove tests + edge cases	2

"As a player, I want equipment buffs to stack with rules so builds feel intentional."

Definition of Ready

- Rod/Boat/Mount data includes bonuses.

Definition of Done

- Multiple sources combine with clear rules (additive vs. capped).
- Conflicts resolved deterministically (e.g., highest wins).
- Unit tests for merging behavior.

Task	SP
Read bonuses from equipment SOs	3
Conflict resolution & caps	5
Tests for merge rules	2

"As a player, I want clear buff icons and tooltips so I know what's active."

Definition of Ready

- Buff events and IDs exist.

Definition of Done

- HUD buff bar with icons, stacks, and time left.
- Tooltip shows description and source.
- Audio ping on gain/expire (subtle).

Task	SP
Icon atlas and mapping to buff ids	2
HUD bar with countdowns	3
Tooltip component	2
Audio cues	1

"As a player, I want debuffs (storm, poison, exhaustion) and ways to cleanse them."

Definition of Ready

- Enemy/weather hooks exist.
- Inventory supports consumables.

Definition of Done

- Debuffs apply from hazards or events.
- Cleansing via items, resting at docks, or time expiry.
- Visual feedback (tint, vignette) during debuff.

Task	SP
Define debuff set + visuals	2
Apply from enemies/weather/events	3
Cleansing methods with UI hints	3

Summary for Epic 13:

User Story	Story Points
13.1 Core Buff System	12 SP
13.2 Time/Biome Buffs	6 SP
13.3 Equipment Buff Stacking	10 SP
13.4 Buff UI & Tooltips	8 SP
13.5 Debuffs & Cleansing	8 SP
Total	**44 SP**

Epic 14 - Character Customization

"As a player, I want to choose my character's look (skin tone, body type, basic outfit) so I feel represented."

Definition of Ready

- Player prefab exists.
- Materials/shader support color tints.

Definition of Done

- Character creator: sliders/selectors for core options.
- Preview updates in real time; confirm applies to in-game avatar.
- Settings persist to save.

Task	SP
AvatarConfig data + defaults	2
Creator UI (sliders/selectors)	3
Apply to prefab/materials & save	5

Persist/load on session start	2

"As a player, I want to unlock cosmetics via quests, shops, or milestones."

Definition of Ready

- Inventory & quest reward flow exist.

Definition of Done

- `CosmeticData` defines category (hat/outfit), rarity, unlock source.
- Unlocks trigger toasts and appear in wardrobe.
- Progress tracked in save.

Task	SP
Create `CosmeticData` SO	2
Unlock rules (quest/shop/achievement)	3
Persistence wiring	2
Rarity/tags for filtering	1

"As a player, I want a wardrobe screen to equip/unequip cosmetics with a live preview."

Definition of Ready

- Cosmetic unlocks exist.

Definition of Done

- Wardrobe grid lists owned items by category.
- Preview mannequin with rotate/zoom.
- Equip/unequip applies instantly to player.

Task	SP
Wardrobe UI grid & filters	3
Equip/unequip pipeline	3
Preview mannequin	2
UI feedback SFX	1

"As a player, I want hats and gear to anchor cleanly to my character without clipping."

Definition of Ready

- Rig/humanoid bones or anchor points defined.

Definition of Done

- Attach points for head/back/hands.
- Hide/show rules (e.g., hide hood under hat).
- Tested with walk/fish/boat animations.

Task	SP
Define anchor points on rig	2

Attachment system + offsets	3
Category rules & masking	3
Animation tests and fixes	2

Summary for Epic 14:

User Story	Story Points
14.1 Character Creator	12 SP
14.2 Cosmetic Unlocks	8 SP
14.3 Wardrobe UI	9 SP
14.4 Attachments & Clipping Rules	10 SP
Total	**39 SP**

Epic 15 - Saving, Loading & Progress System

"As a developer, I want a robust save framework with profile slots so players can manage runs."

Definition of Ready

- Core systems expose serializable state.

Definition of Done

- Save interface abstracts JSON/binary implementation.
- Multiple profile slots with create/delete/rename.
- Basic obfuscation and integrity checks.

Task	SP
Save format & interface	3
Slots directory & metadata	2
Lightweight obfuscation/encryption	2
Serialization unit tests	3

"As a player, I want the entire game state saved (inventory, quests, time, location, portals, notes)."

Definition of Ready

- Systems have read/write accessors.

Definition of Done

- Save captures: inventory, equipment, stats, quests, events, world time/season, player position, portals, map markers & notes.
- Load restores everything reliably.

Task	SP
Serialize inventory/equipment	3

Serialize player/location/time/season	3
Serialize quests/events/bounties	5
Serialize portals/markers/notes	5

"As a player, I want manual save/load and autosave so I never lose progress."

Definition of Ready

- Save framework functional.

Definition of Done

- Pause menu: Save/Load with slot selection.
- Autosave on key triggers (resting, fast travel, quest complete).
- Save icon/spinner + toasts; safe pause during save.

Task	SP
Save/Load UI	3
Autosave scheduler + triggers	3
Save indicator + toasts	1
Gameplay lock during save I/O	2

"As a developer, I want versioning and migration so future updates don't break saves."

Definition of Ready

- Save format defined.

Definition of Done

- Version field stored per save.
- Migration pipeline for older versions.
- Checksums & error handling with fallback.

Task	SP
Add versioning & migration stubs	3
Checksum/integrity verification	3
Error handling, rollback, fallback	3

Summary for Epic 15:

User Story	Story Points
15.1 Save Framework & Slots	10 SP
15.2 Full Game State Save	16 SP
15.3 Manual/Autosave	9 SP
15.4 Versioning & Integrity	9 SP
Total	**44 SP**

Epic 16 - UI & User Experience

"As a player, I want a cohesive UI theme and smooth transitions so the game feels polished."

Definition of Ready

- UI Canvas and prefabs in use.

Definition of Done

- Shared UI style guide (fonts, spacing, buttons).
- Common components (panels, lists, toasts) as prefabs.
- Simple transitions and sounds across major panels.

Task	SP
Codify style guide & variables	2
Common UI prefabs (Panel/List/Toast)	3
Transitions/animations	3
Global UI SFX	1

"As a player, I want accessibility options (scalable text, colorblind-safe icons) so I can tailor readability."

Definition of Ready

- Settings menu exists.

Definition of Done

- Text scale slider (persists).
- Colorblind palettes/icons for rarity and buffs.
- High-contrast mode toggle.

Task	SP
Font scaling system	2
Colorblind-safe icon/palette set	3
Contrast adjustments + toggle	2
Settings persistence	1

"As a player, I want full controller navigation and input glyphs in the UI."

Definition of Ready

- Input Actions set up.
- Keyboard controls already work.

Definition of Done

- Navigate UI with gamepad (focus system).
- Glyphs adapt to current device (KB/mouse/gamepad).

- Remapping page shows correct prompts.

Task	SP
Device glyph mapping	3
Gamepad navigation & focus	5
Remap screen with live prompts	3

"As a player, I want an onboarding tutorial with contextual tooltips so I learn while playing."

Definition of Ready

- Core loops (fish, move, map) exist.
- Quest and tooltip UI available.

Definition of Done

- First-run tutorial quest chain guiding movement, map, basic fishing, inventory.
- Contextual tooltips appear on first use and can be muted.
- "Help" panel lists basics.

Task	SP
Outline tutorial steps & triggers	2
Tooltip system with suppression flags	3
Implement tutorial quest chain	5

Summary for Epic 16:

User Story	Story Points
16.1 UI Theme & Transitions	9 SP
16.2 Accessibility Options	8 SP
16.3 Controller UI Support	11 SP
16.4 Onboarding & Tooltips	10 SP
Total	**38 SP**

Epic 17 - Voxel Modeling & Art Style

"As a developer, I want a modular voxel asset pipeline so art stays consistent and fast to iterate."

Definition of Ready

- Basic art import path works.

Definition of Done

- Folder structure, naming, and prefab conventions documented.
- Import presets for scale/pivots/colliders.
- Style guide with palette and proportions.

Task	SP
Folders & naming templates	1

Import presets (scale/pivot/collider)	1
Prefab conventions (variants, LOD anchors)	2
Style guide document	2

"As a player, I want distinct fish models with visible mutations and size variants."

Definition of Ready

- Fish catching loop + DB exist.

Definition of Done

- Base fish model + size scaling rules (Tiny→Monster).
- Material variants per mutation (glow, tint, particles).
- UI thumbnails for log and catch popups.

Task	SP
Base fish model & rig (if needed)	3
Mutation materials/VFX set	3
Size scaling rules per size class	2
Thumbnail renders for UI	2

"As a player, I want biomes to have unique tilesets and props to make exploration visually rewarding."

Definition of Ready

- Biome list is defined.

Definition of Done

- Tileset + prop pack for at least 3 biomes (e.g., Beach, Swamp, Volcanic).
- Ambient elements (fog/particles) aligned with style.
- Lighting presets per biome.

Task	SP
Create 3 biome tilesets	5
Ambient prop pack	3
Lighting presets per biome	2

"As a developer, I want performance-minded LODs and batching, so voxel scenes run smoothly."

Definition of Ready

- Profiling tools available.

Definition of Done

- LODs/impostors for heavy meshes.
- Static batching / mesh combine where possible.
- Profile meets target frame rate on test scene.

Task	SP
Implement LODs/impostors	5
Static batching / mesh combine passes	3
Profiling & optimization fixes	3

Summary for Epic 17:

User Story	Story Points
17.1 Voxel Asset Pipeline	6 SP
17.2 Fish Models & Mutations	10 SP
17.3 Biome Tilesets & Props	10 SP
17.4 Performance & LODs	11 SP
Total	**37 SP**

Epic 18 - Future Expansions & DLC

"As a developer, I want a DLC/content-pack framework (cosmetic-first, no pay-to-win) so I can ship optional content safely."

Definition of Ready

- Save/versioning exists.
- Boat/cosmetic systems exist.

Definition of Done

- AssetBundle (or Addressables) loading wrapper.
- DLC toggle per profile (enabled/disabled) with fallback to defaults.
- Content gating that doesn't break saves when disabled.

Task	SP
DLC toggle + persistence	2
AssetBundle/Addressables wrapper	5
Content gating & graceful fallbacks	3

"As a player, I want a Flying Dutchman cosmetic boat skin as a showcase DLC."

Definition of Ready

- Boat system supports skins and VFX.

Definition of Done

- Flying Dutchman skin with subtle spectral VFX and wake.
- Unlock flow (quest completion or cosmetic pack).
- Skin is cosmetic-only; stats unchanged.

Task	SP
Boat skin asset + spectral VFX	3

Unlock flow (quest or purchase)	3
Verify stat neutrality & compatibility	2

"As a player, I want lighthouses seeded in the world (teaser system) to foreshadow a future lighthouse feature."

Definition of Ready

- World spawn system exists.

Definition of Done

- Lighthouse prefab spawns at curated coastal points.
- Map icons + short lore notes via NPCs.
- Hooks in place for future questline.

Task	SP
Prefab + spawn rules	3
Map icon + lore note integration	2
Future quest hooks (no logic yet)	1

"As a developer, I want feature flags and a feedback loop so I can iterate with players."

Definition of Ready

- Settings and save frameworks exist.

Definition of Done

- Feature flags (enable/disable experimental features).
- A/B-ready config channel in memory/save.
- Feedback panel with exportable logs (opt-in).

Task	SP
Feedback panel + optional logs	1
Feature flag system	5
A/B config & persistence	3

Summary for Epic 18:

User Story	Story Points
18.1 DLC Framework (Cosmetic-first)	10 SP
18.2 Flying Dutchman Skin	8 SP
18.3 Lighthouse Teaser System	6 SP
18.4 Feature Flags & Feedback	9 SP
Total	**33 SP**